



University
of Basel

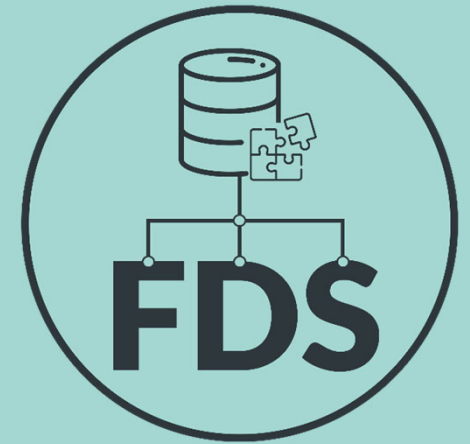
Chapter 4: Data

4.1: Self-Organization (P2P)

Foundations of Distributed Systems

Fall Semester 2025

Prof. Dr. Heiko Schuldt



Overview of Chapter 4

4.1	Self-Organization (P2P)
4.1.1	What is a Peer-to-Peer System?
4.1.2	P2P Content Sharing
4.1.3	Distributed Hash Tables
4.2	Data Consistency
4.3	Data Replication

Objectives of Chapter 4.1 – Self-Organization

- Traditional system architectures introduce dedicated roles (client vs. server) and usually have some centralized component(s) and thus some form of single point of failure, especially when it comes to data management (e.g., a centralized database system).
- Self-organizing (peer-to-peer) systems are an alternative approach and are specifically designed to avoid any form of centralization.

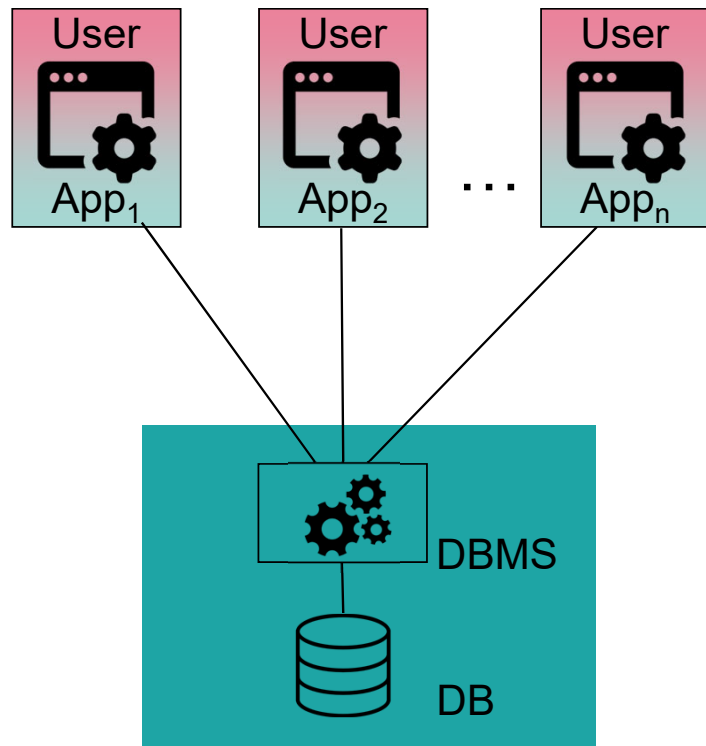
You will ...

- understand the different generations of peer-to-peer data management and data sharing systems,
- learn the concepts of distributed hash tables for data management,
- become familiar with selected second generation peer-to-peer data structure using distributed hash tables.

Overview of Chapter 4.1 – Self-Organization

4.1	Self-Organization (P2P)
4.1.1	What is a Peer-to-Peer System?
4.1.2	P2P Content Sharing
4.1.3	Distributed Hash Tables
4.2	Data Consistency
4.3	Data Replication

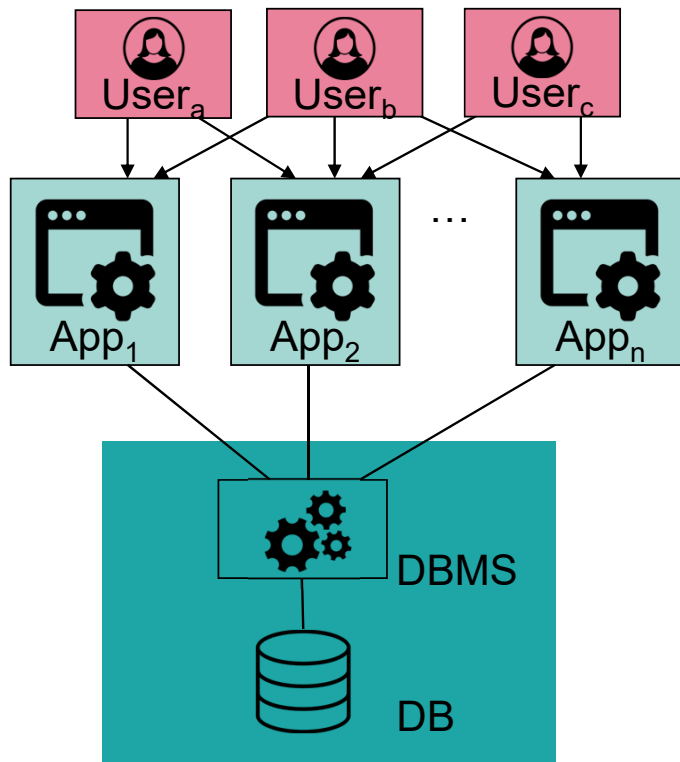
Basic Architectural Paradigms: Client/Server



Two-tier Client/Server-Architectures

- Applications and DBMS in different address spaces. Each application (App_i) corresponds to a single user
 - Role of the DBMS
 - Platform for clients that concurrently access shared data
 - Requirements:
 - Data definition
 - Data access and manipulation
 - Transactions
 - In addition, further functionality is transparently provided for users / apps:
 - Query optimization, load balancing, scheduling, parallelization, correctness of concurrent access, recovery, persistence, availability, efficiency, ...
- Data independence

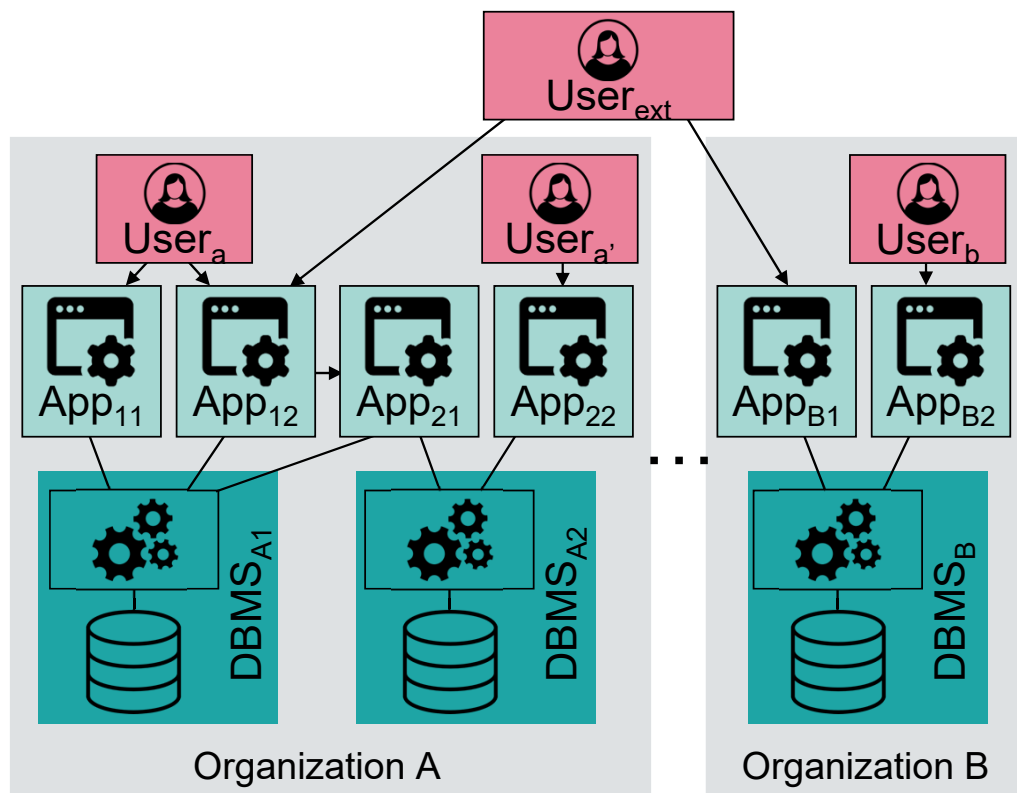
Basic Architectural Paradigms: Multi-tier Systems



- Decoupling of application logic and user interfaces
 - Each application (App_i) can be employed by multiple users ($User_k$)
- Service-orientation
- A user can use (via his/her GUI) several services – however independent of each other
 - There is (per se) no way to combine services within a logical entity (transactions)

Three-tier Client/Server-Architectures

Basic Architectural Paradigms: Distributed IS



Multi-tier Client/Server-Architectures

- Extension of the service-orientation paradigm
→ **Distribution**
- Services are used even across system boundaries (Heterogeneity)
 - Integration of services
- Traditional role of a DBMS is now rather inferior
 - The management of services at the fore
- **Infrastructure for the integration of services across system boundaries (and within a transaction context) required!**

DBMS

What is a Peer-to-Peer System?

- In general, peer-to-peer (P2P) is an architectural paradigm where control/data/... is shared among a set of equal peers (which means that each peer is at the same time client and server)
 - In general terms: distributed resources are shared and exploited by direct communication between the peers
 - The difference between P2P and Client/Server is that in the former, the roles 'client' and/or 'server' are not fixed and also not static – thus can be changed dynamically
 - “self-organized system of equal, autonomous systems without centralized infrastructure based on an unreliable service”
- Mostly, P2P is associated with content (file) sharing, more recently also with distributed data management in a more general setting
- However, P2P is more than data management and addresses all kind of distributed infrastructures / control

Overview of Chapter 4.1 – Self-Organization

4.1	Self-Organization (P2P)
4.1.1	What is a Peer-to-Peer System?
4.1.2	P2P Content Sharing
4.1.3	Distributed Hash Tables
4.2	Data Consistency
4.3	Data Replication

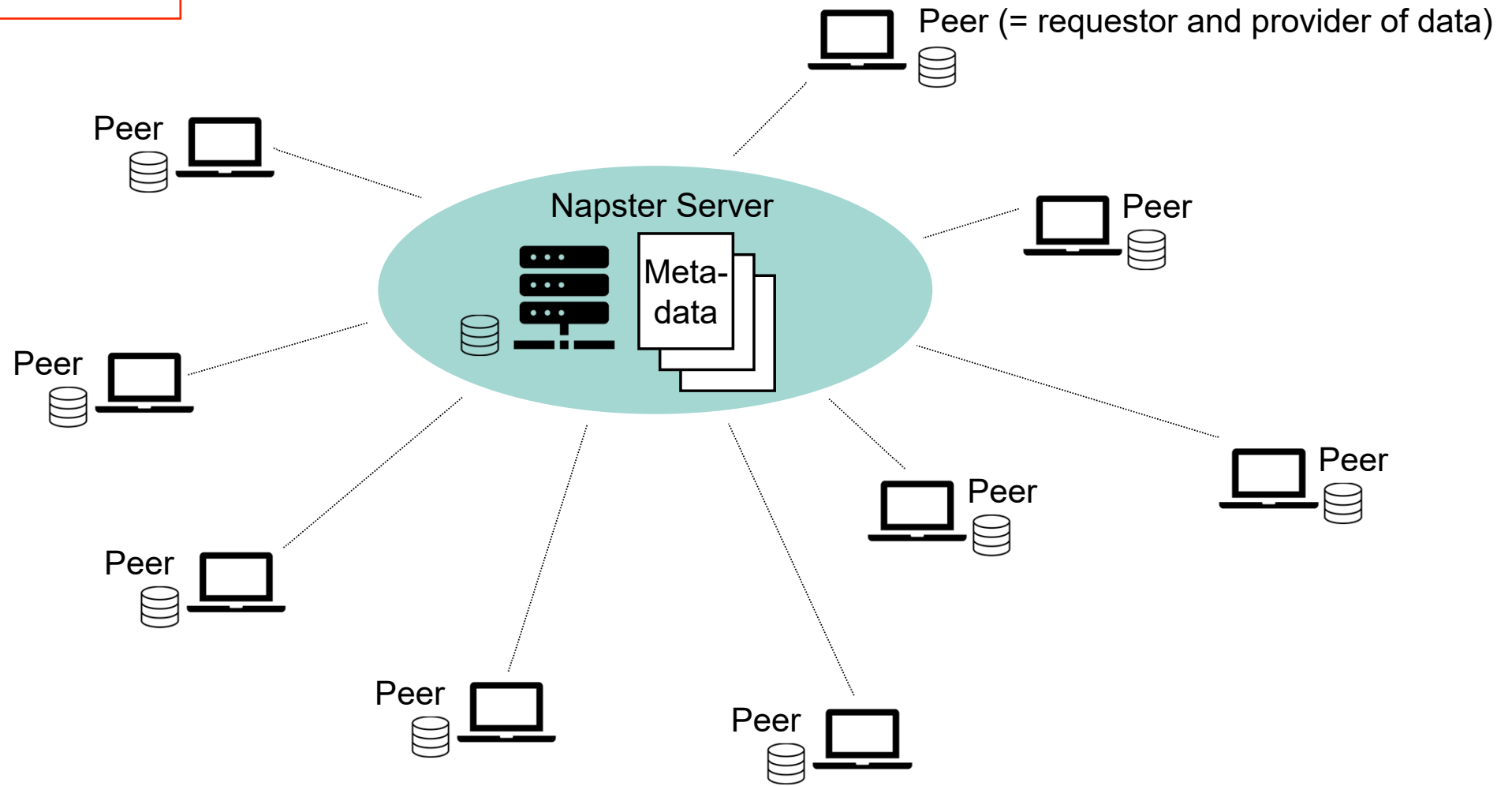
P2P Data Management ('Content Sharing')

- One of the most common applications of P2P systems is the management of large volumes of data and the support for efficient and effective search
- Consequences of distributed control and data management:
 - Systems are highly scalable
 - They are not vulnerable to censorship
- P2P networks of the first generation mostly focus on content sharing
 - Napster
 - Gnutella
 - KaZaA
- Many P2P data structures of the second generation explicitly address data management in a self-organizing network (i.e., a network which automatically adapts to the existing peers)
 - Usually based on Distributed Hash Tables (DHT)

Napster ...

- First “Peer-to-Peer” network
- Has been designed for file sharing applications (access to shared documents or data, respectively)
 - First version: June 1999, developed by Shawn (Napster) Fanning
 - Quickly evolved to a platform for exchanging audio files
 - In 2000, the music industry has sued Napster (because of distribution of copyrighted material)
 - End of 2000, Napster has evolved to a commercial file sharing platform (under the roof of Bertelsmann E-Commerce)
- Napster: network topology
 - At first glance, Napster follows a traditional client/server architecture
 - The Napster server manages
 - An **index with metadata** of the entire system (file name, date, etc.)
 - A **table with all connection details** of participating clients
 - A **table with the files** of all the participating clients
 - **But NOT the actual data**

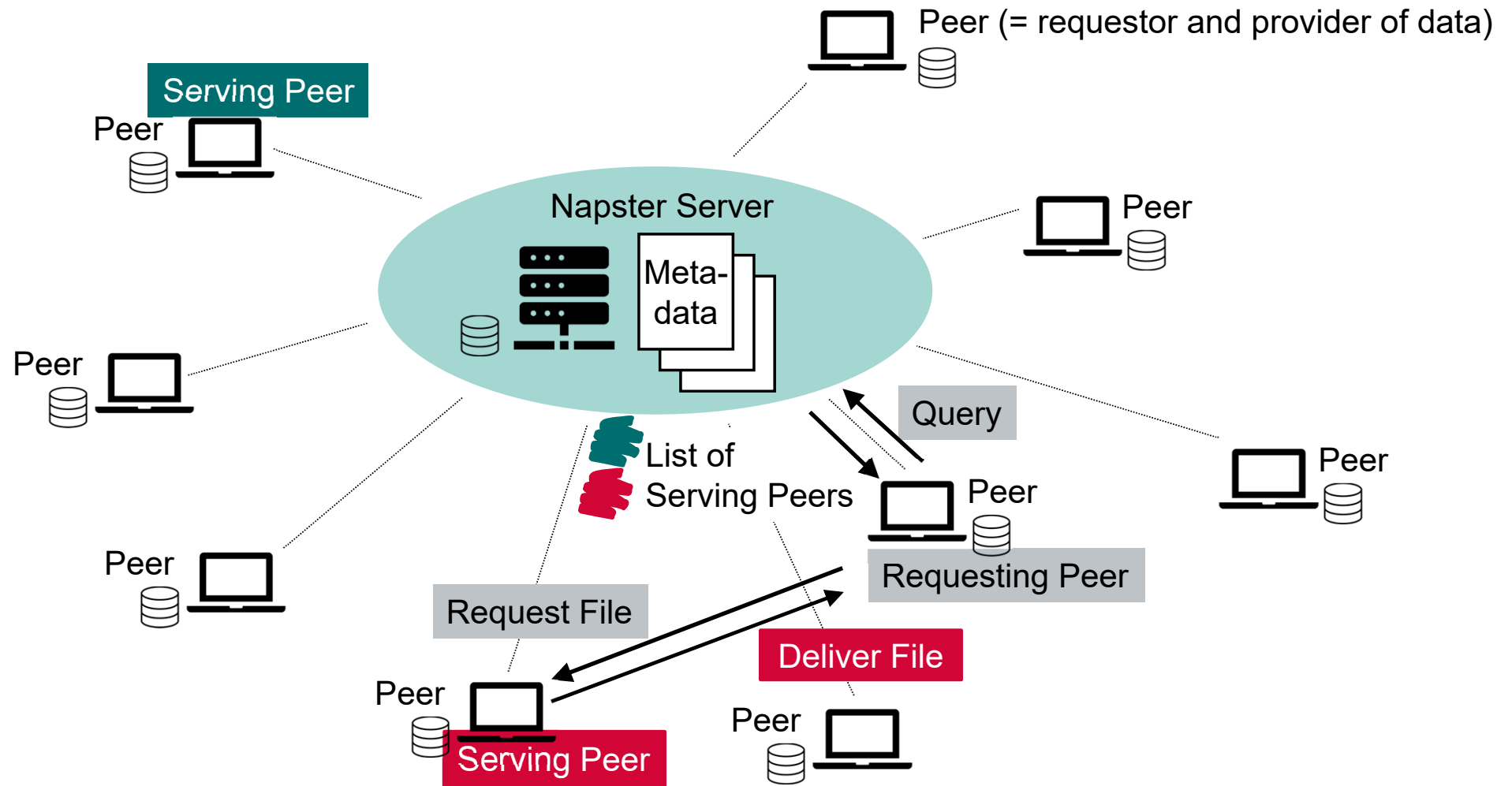
... Napster ...



... Napster ...

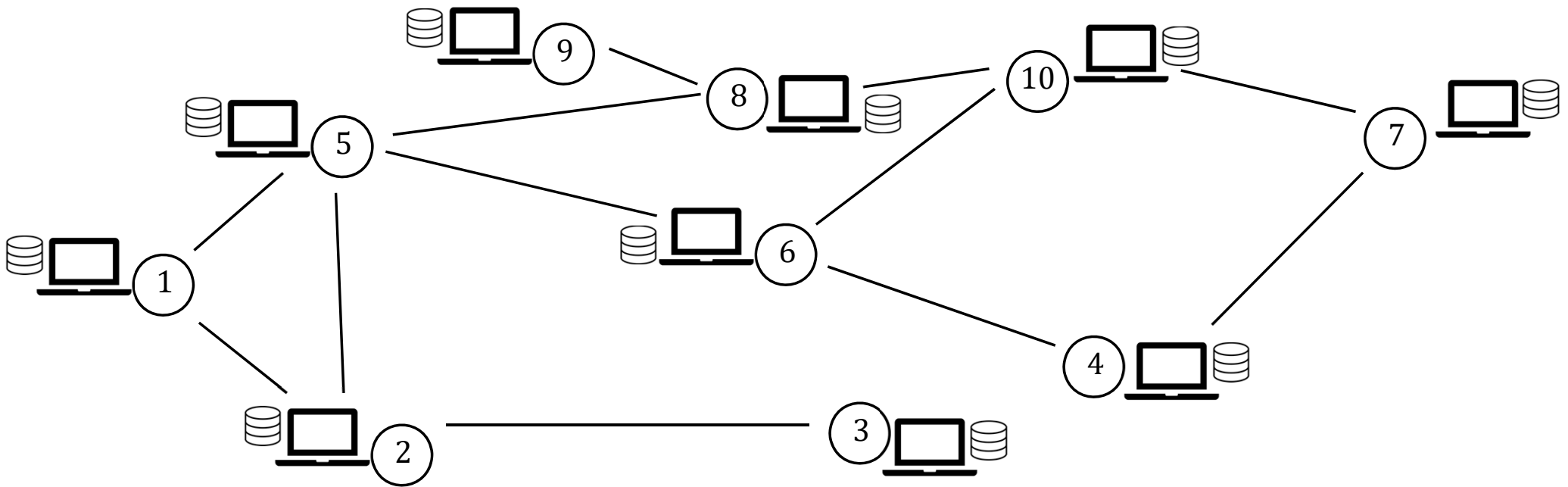
- Query processing in Napster
 - Client asks the server for a file with a given name
 - The server checks locally (in its metadata tables) for Napster peers that have this file
 - The server returns to the client a list of Napster peers that provide this file
 - The client directly contacts one of these peers and downloads the file from there (this is the only true P2P interaction in Napster)
- Properties of Napster
 - + Network topology is simple (since rather conventional)
 - + Files can be located and found quickly, in one hop (at the server's site)
 - The centralized architecture implies a „single point of failure“ and also gives room for censorship
 - Due to the centralized server, Napster does not scale (i.e., it cannot support an ever increasing number of peers)
 - Performance goes down with increasing number of peers
 - # of files / # of peers limited by the storage available at the server's site

Napster ...



gnutella ...

- gnutella has been designed for file sharing applications as well
 - First version: March 2000, developed by Justin Frankel and Tom Pepper
 - In contrast to Napster, **gnutella does not have any single central component**



... gnutella ...

Gnutella's architecture

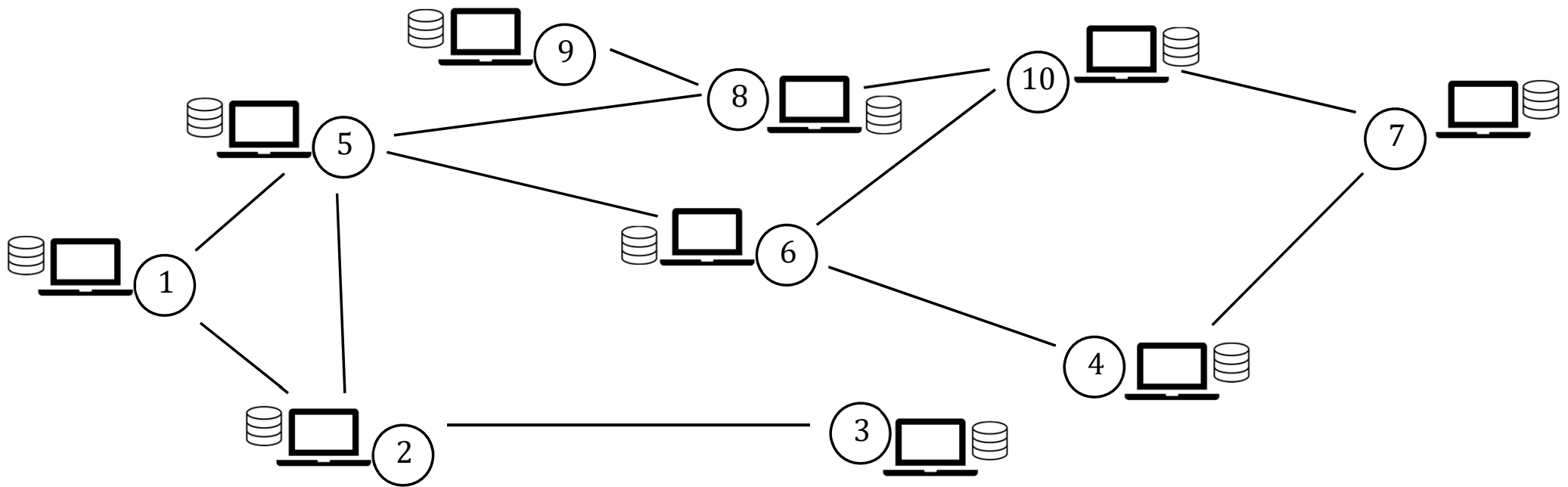
- In gnutella, peers are directly linked to other peers, thereby forming a network of peers
- This structure is established dynamically
 - When downloading the gnutella software, it comes up with an initial list of peers; peers from this list are contacted until at least one is found which is active
 - An active peer forwards its neighborhood list to the peer which is to be added to the network
 - This neighborhood list is continuously updated and maintained along with the interaction of a peer with others in the network

... gnutella ...

- The protocol for adding a new peer consists of two phases
 - Phase 1 „ping“
 - A new gnutella participant N issues a query for neighborhood lists
 - This query is provided with a TTL information (TTL = time to live)
 - The query is forwarded to neighbors, they forward it to their neighbors, etc. (flooding algorithm)
 - The TTL information prevents that a query stays in the network forever such that already few queries can lead to a high load in the network. Whenever a query is forwarded, its TTL value is decremented; a peer only forwards a query when $TTL > 0$
 - Phase 2 „pong“
 - As reaction to the ping, neighborhood lists of the peers that have been reached are returned via the request path
 - These lists include the IP addresses and ports of the peers found

... gnutella ...

- How a new peer (#11) is integrated into the network

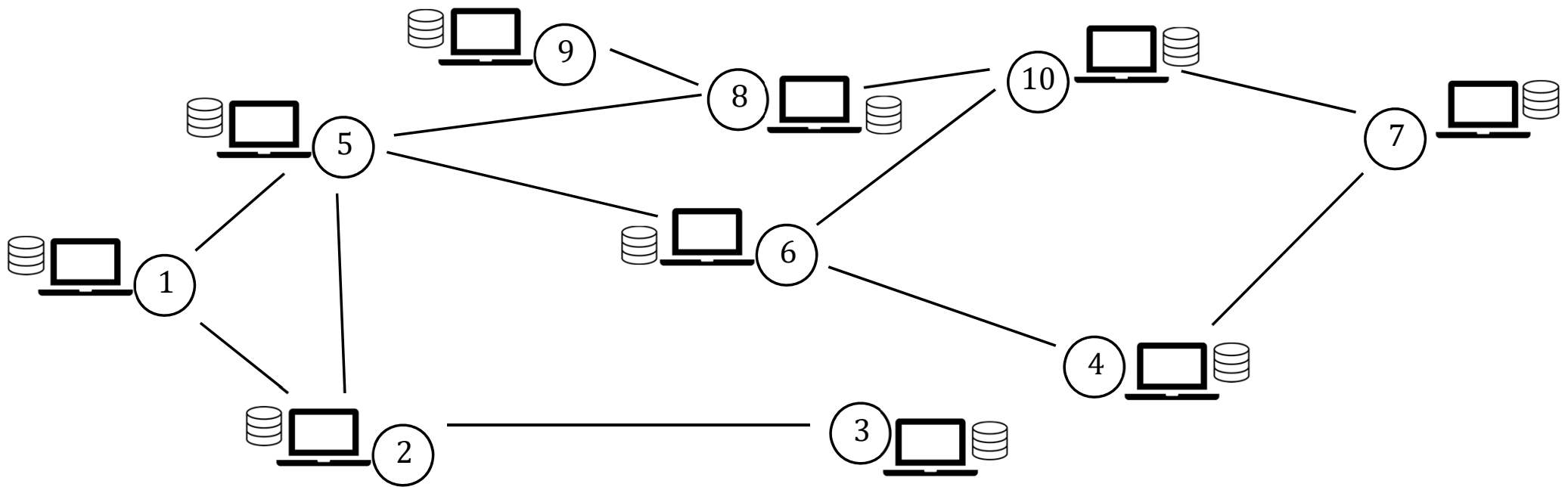


... gnutella ...

- **Searching a file** also is done in two phases
 - Phase 1: „Query“
 - The query with a filename (and attached TTL value) **is forwarded to all known neighbors**
 - These process it locally and also forward it with decremented **TTL value** (only if $TTL > 0$)
 - Phase 2: „Query hits“
 - The response **is returned via the same path**, in reverse order, to the peer which has issued the query
 - If the file is available with one or several peers, the peer which has issued the query picks one address and starts a direct download

... gnutella ...

- Query processing in gnutella



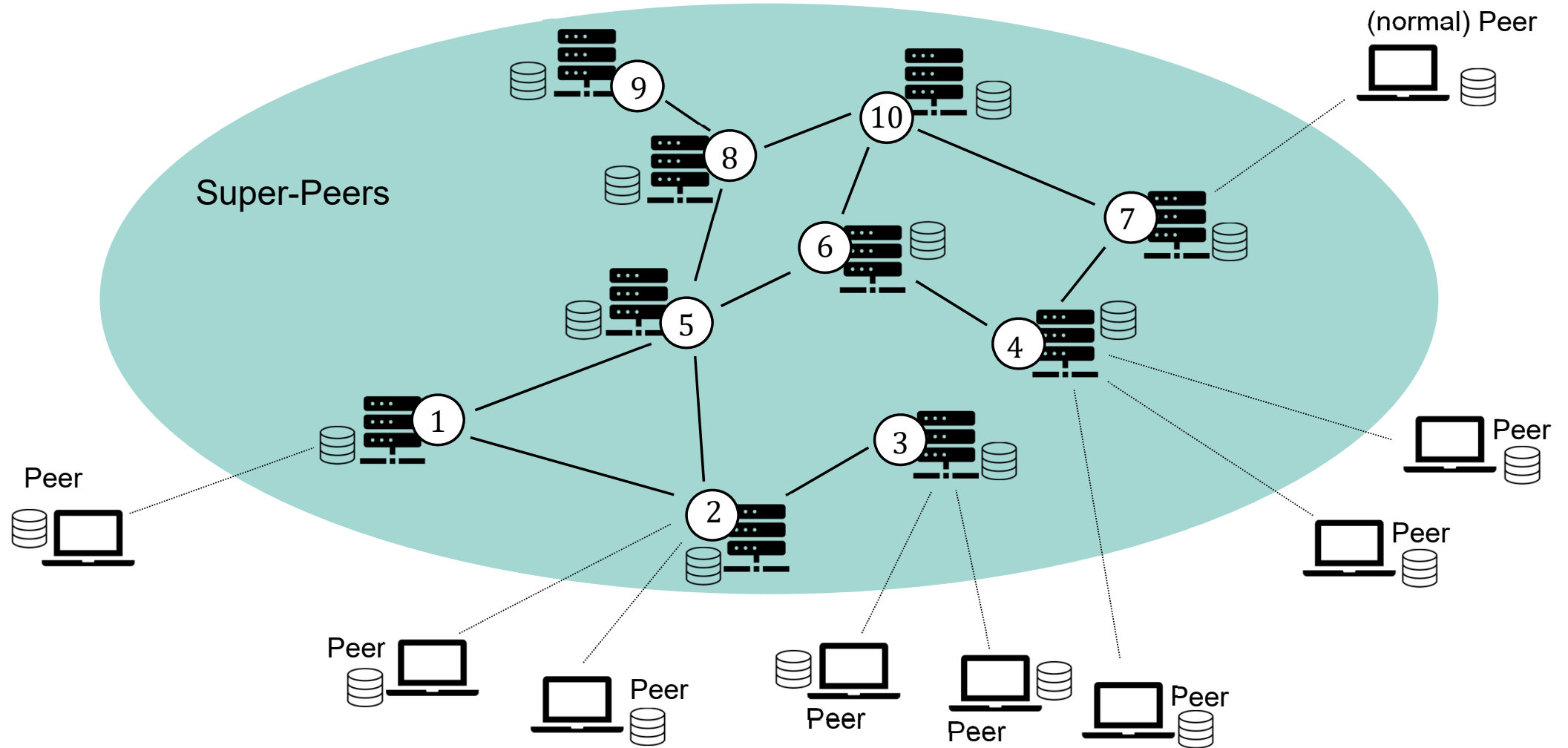
... gnutella

- Properties of gnutella
 - + Distributed network infrastructure, no centralized component, no single point of failure
 - Due to TTL, an implicit network partitioning takes place for queries (this might have consequences on the success rate)
 - High latency

Super Peer Networks (Example: KaZaA) ...

- KaZaA (and other similarly structured P2P networks) have a **hybrid structure which combines aspects of Napster and gnutella**
 - Nodes with high bandwidth connection are selected as “**super peers**”; the network topology is also termed **Super-Peer-Network**
 - Super peers maintain a P2P network in gnutella style
 - „Normal peers“ are connected as clients to a super peer; the latter can be considered, from the point of view of their normal peers, as similar to the Napster server

... Super Peer Networks (Example: KaZaA)



Super Peer Networks

- Properties of super peer networks
 - + Better scalability compared to Napster while
 - + Having lower latency as gnutella (shorter paths in the network)
 - What happens when for instance the available bandwidth of a super peer changes?
 - i.e., it is quite vulnerable to changes in the environment

Characterization of P2P Networks ...

A short excursion to graph theory

- The degree d_n of a node n in a graph G is the number of edges incident to this node.

The degree d of a graph G is defined as the maximum number of edges incident to a node of the graph, i.e. $d = \max_{n \in G}(d_n)$

A graph is k -connected if

- it has more than k edges
- it always remains connected whenever fewer than k edges are removed

- In a k -connected graph,
 - you can always remove $k-1$ edges, arbitrarily chosen, and it remains connected
 - after the k^{th} edge is removed, the graph is no longer connected but is rather split in two or more unconnected sub-graphs

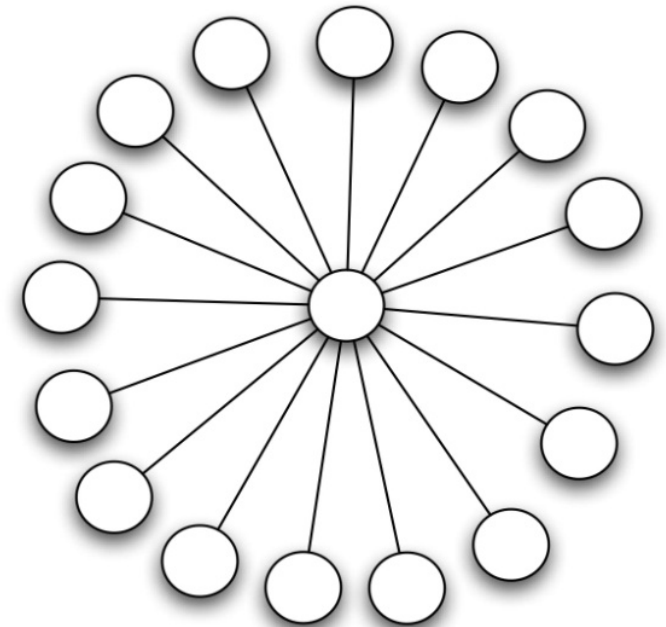
... Characterization of P2P Networks ...

Napster

- Follows a centralized architecture („star graph“)
- Degree of the graph: $d_{\text{Nap}} = n-1$ (with n : #of peers)
- The star graph is 1-connected (i.e., $k_{\text{Nap}}=1$)

Therefore, Napster ...

- **does not scale well**, since the **degree of the graph is too high** (bottleneck in communication)
- **is not robust** enough since the **connectivity is too low**



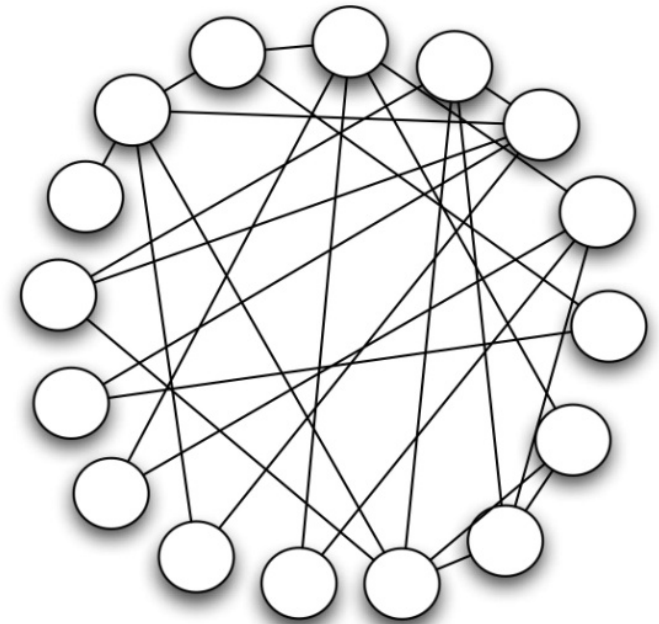
... Characterization of P2P Networks ...

gnutella

- Follows a completely decentralized architecture (arbitrary connections in the graph)
- The graph has a low degree (few neighbors), i.e., $d_{\text{Gnu}} < d_{\text{Nap}}$
- The connectivity is higher than the one of Napster i.e., $k_{\text{Gnu}} > k_{\text{Nap}}$ (concrete value depends on size of neighborhood list)

Therefore, gnutella ...

- is **very robust** since the **connectivity is high**
- does **not necessarily scale well**, since in worst case the **complete network** has to be scanned in order to make sure that a particular data is found



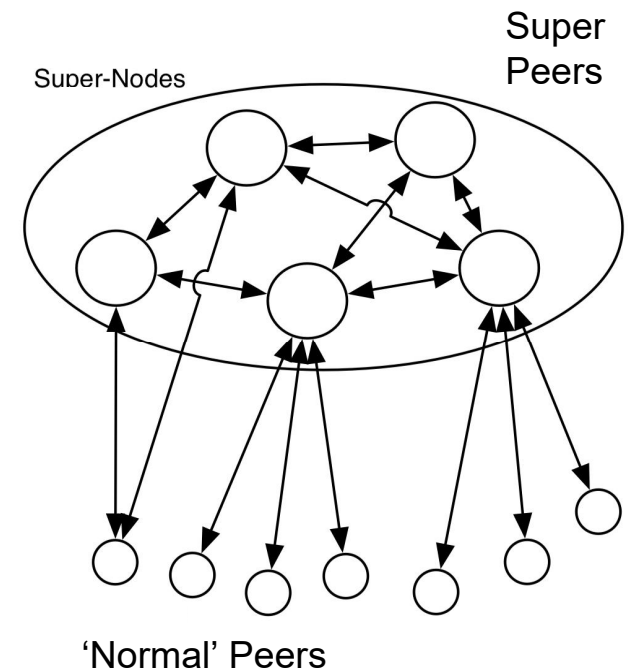
... Characterization of P2P Networks

Super Peer Architectures (like KaZaA)

- Architecture in parts centralized
- The **degree of the graph is low** ('normal' peers only have one neighbor; super peers just a few other super peers)
i.e., $d_{\text{Gnu}} < d_{\text{Kaz}} < d_{\text{Nap}}$
- The connectivity is the one of Napster ($k_{\text{Kaz}} = 1$), but it can be increased (> 1) when a 'normal peer' connects to several super peers

Therefore, Super Peer Architectures ...

- **Scale better than gnutella or Napster**, due to separation in super peers and 'normal peers' (queries do not have to be propagated through the complete network and are also not handled by a central node)



Important Issues in P2P Data Management ...

- Important Issues in P2P Data Management
 - **Where** can a particular data item be found (or at least information on it)?
 - **How** can I find the way from my current peer to the one hosting the data I am looking for?
- The Napster approach:
 - Where? Information available on a central server
 - How to get there? Direct connection
- The gnutella approach:
 - Where? Not known a priori
 - How to get there? Ask all nodes in the network (unstructured query routing)

... Important Issues in P2P Data Management

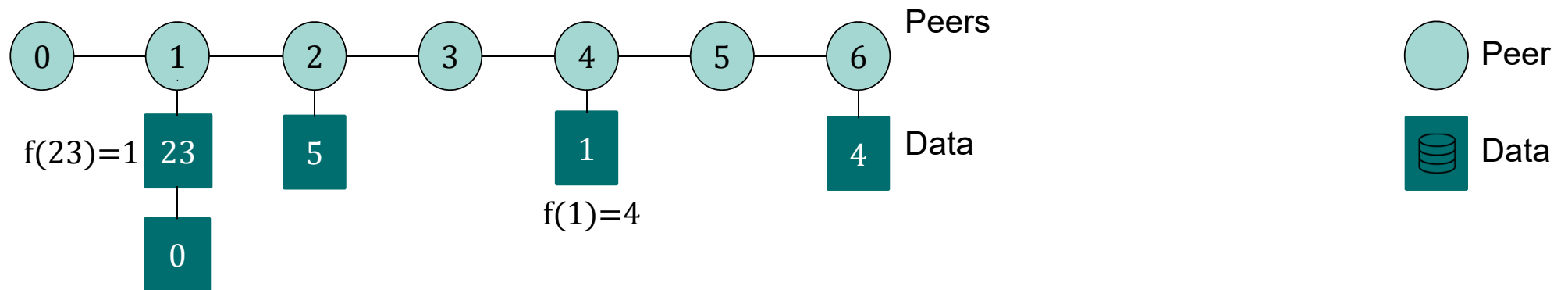
- Is there a better solution?
 - Where is data item x ? at location $f(x)$, where $f(x)$ is a function known to all nodes in the network
 - How to get there? via a route that shows me the way from my current location to $f(x)$; usually, just the next node in direction to $f(x)$ is known, more information is available there

Overview of Chapter 4.1 – Self-Organization

4.1	Self-Organization (P2P)
4.1.1	What is a Peer-to-Peer System?
4.1.2	P2P Content Sharing
4.1.3	Distributed Hash Tables
4.2	Data Consistency
4.3	Data Replication

Hash Table as Peer-to-Peer Network

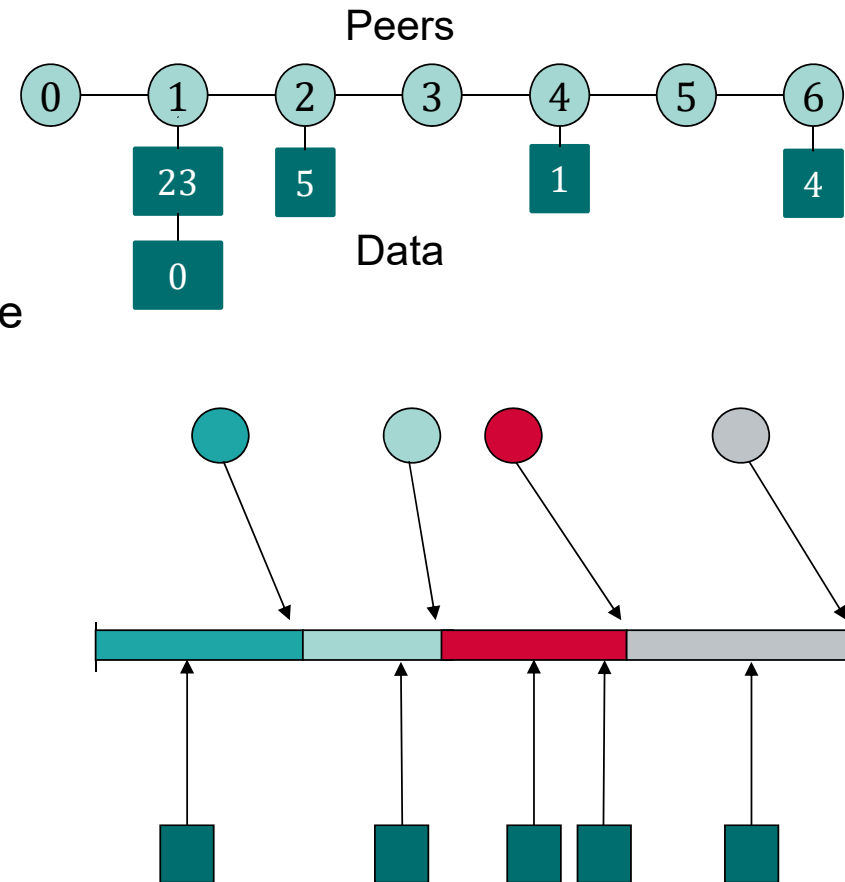
- Basic Idea: each peer of the network represents a bucket of a hash table (0, 1, 2, ..., n-1)
 - A hash function known to all peers is applied e.g., for $n = 7$: $f(x) = (3x+1 \bmod 23) \bmod 7$
 - The peers are connected as a (circular) linked list



- Search
 - Step 1: determine $f(x)$
 - Step 2: go to peer with address $f(x)$ along the chain of peers

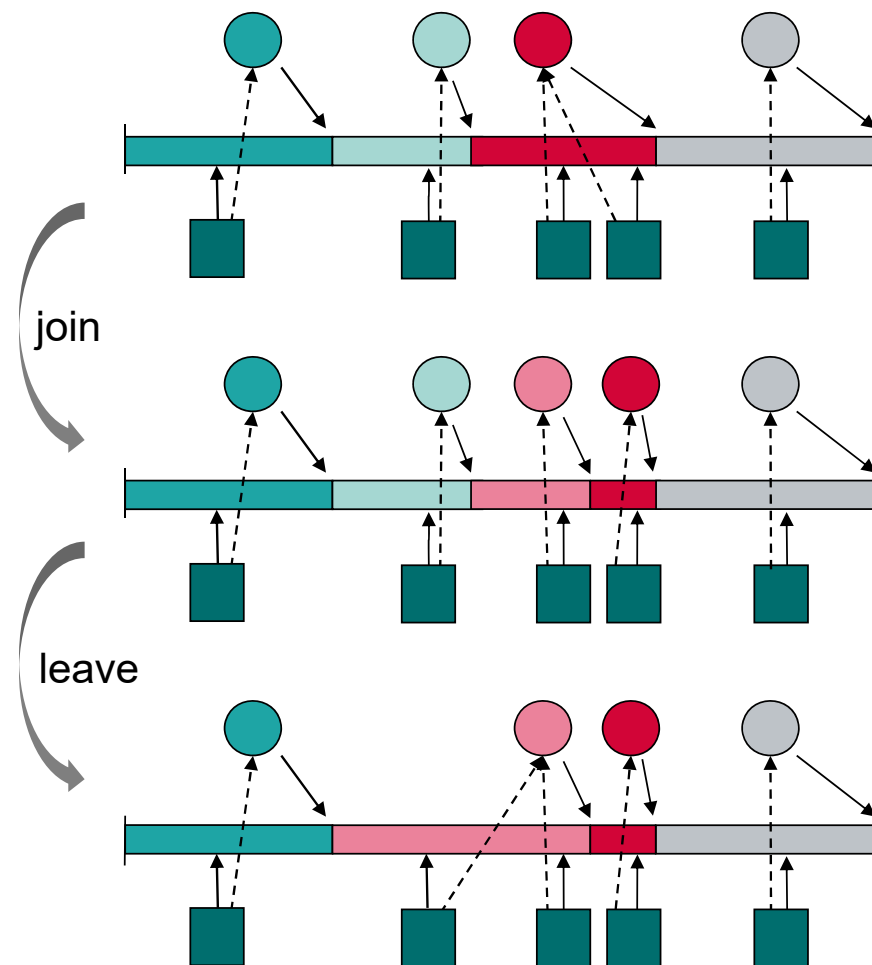
From Hash Tables to Distributed Hash-Tables (DHT)

- Properties of „conventional“ Hash Tables (linked peers)
 - + Search can be implemented easily
 - When a new peer joins, either a new hash function has to be chosen (dynamic extension) or a complete re-hash is necessary
- Distributed Hash-Table: Hashing is applied twice:
 - Peers are hashed to a certain interval of the co-domain of the hash function
 - Similarly, data items are also hashed. They are assigned to a point in the co-domain and thus transitively to the peer responsible for their hash value



DHT: What to do when Peers join or leave?

- If a new peer joins, then the range for which it will be responsible for is determined
- All data items in this interval are taken from the neighbors
- When a peer leaves the network (in a controlled way), then all data items in its range have to be re-distributed to its neighbors
- For this, again the hash function on peers is applied

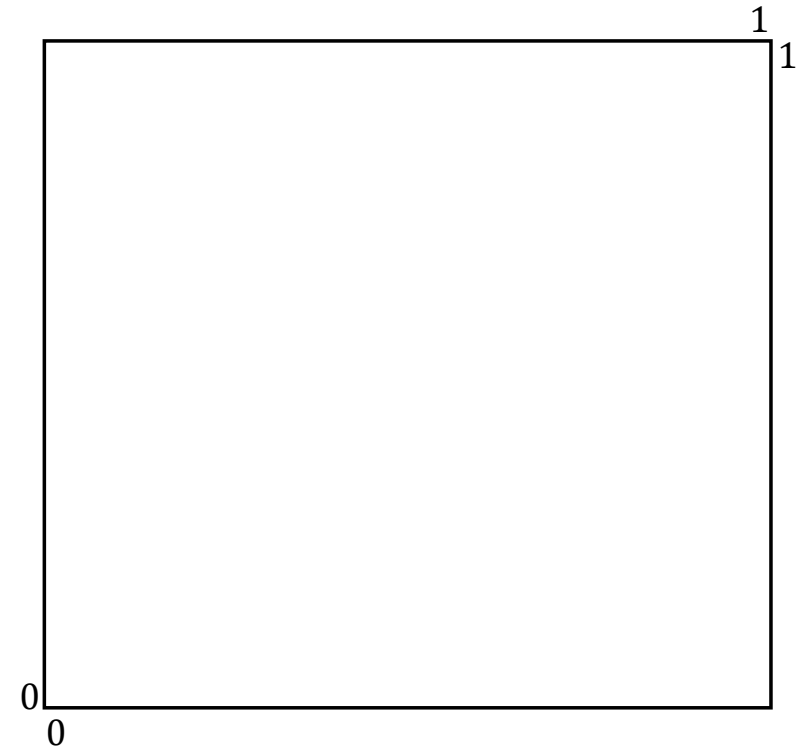


P2P Data Structures (DHTs)

- CAN
- P-Grid
- CHORD
- ... any many more ...

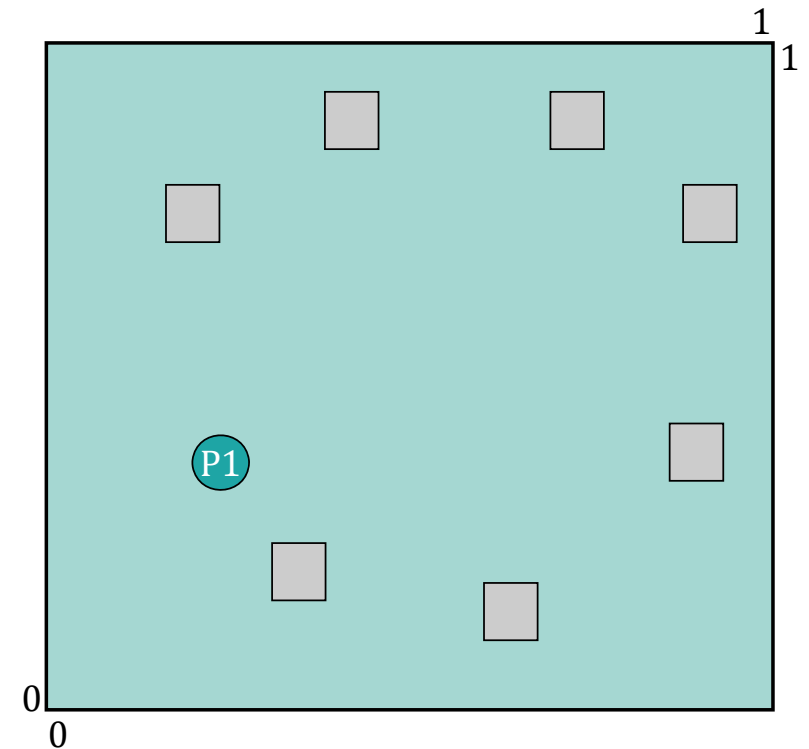
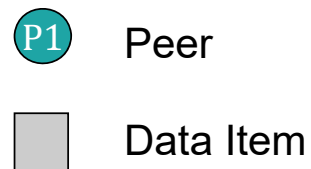
Content Addressable Network (CAN) ...

- Idea: data items are mapped to a d-dimensional address space (in what follows, $d = 2$)
- Data items consist of $\langle \text{key}, \text{value} \rangle$ -pairs
 - Key: calculated by hash function
 - Determines where (logically) the value has to be stored
 - In our example with $d=2$ the hash function returns a point (x and y coordinates)
 - Value: payload (information to be stored)
- Initially, the address space is empty



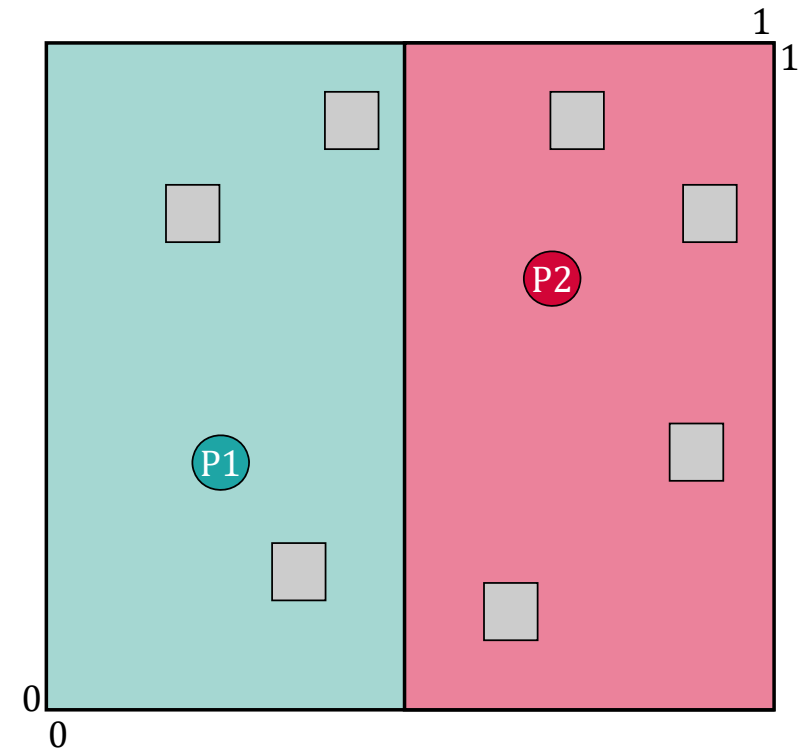
... Content Addressable Network (CAN) ...

- The first peer (P1) is responsible for the complete space ...
- ... thus has to manage all data objects
 - Each data object is mapped to a point in the space by the hash function applied to its value



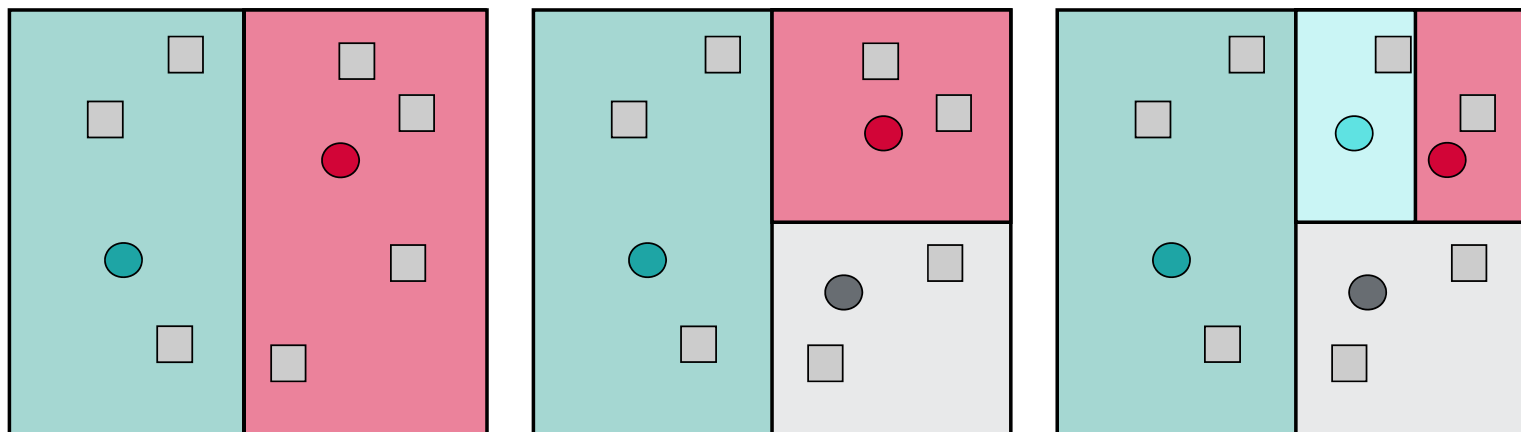
... Content Addressable Network (CAN) ...

- When a new peer enters the system, it randomly chooses a point in the space
- The (formerly) responsible peer has to split the area it is responsible for along one dimension
 - One half remains with the former owner (also the data contained in this area)
 - The second half (including contained data) is moved to the new peer
 - In worst case, one of the peers has to move its representation (point in the d-dimensional space) to 'stay' in its area



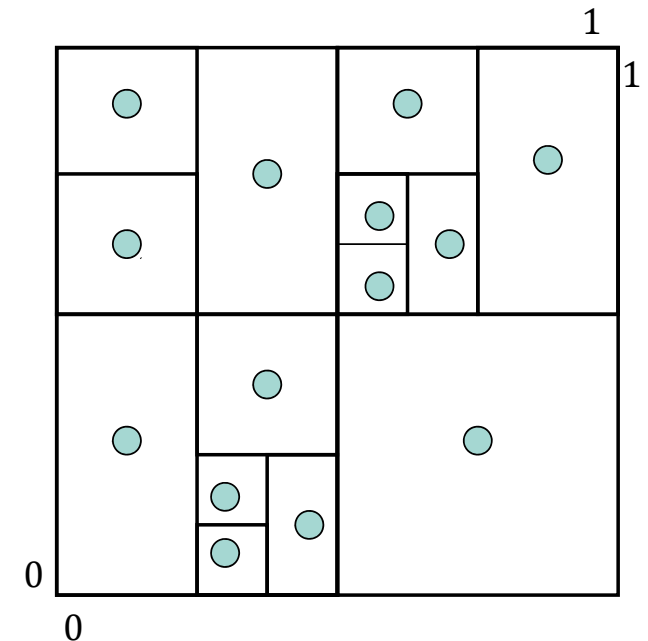
... Content Addressable Network (CAN) ...

- This re-distribution is repeatedly applied whenever new peers join
 - In order to make sure that the areas do not degenerate, the dimensions along which the area is split are usually chosen in alternating order
- If peers leave, this is done in reverse order (areas are merged and data objects are assigned to the responsible peer of the area)



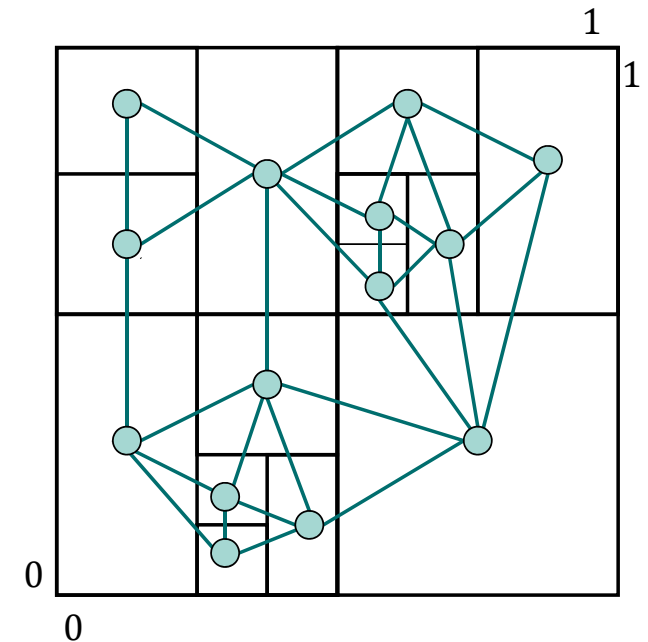
... Content Addressable Network (CAN) ...

- What about the P2P Network?
- Each peer is responsible for an area in the d-dimensional space
- The overlay network is established as follows
 - Each peer is represented by a node
 - An edge is added between each pair of peers which represent adjacent areas
 - Neighborhood information is stored by a peer in a **routing table**



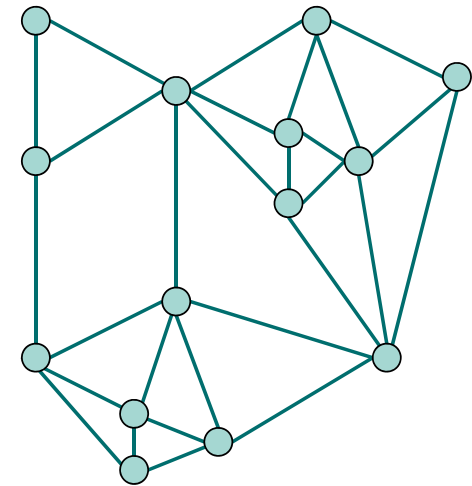
... Content Addressable Network (CAN) ...

- What about the P2P Network?
- Each peer is responsible for an area in the d-dimensional space
- The overlay network is established as follows
 - Each peer is represented by a node
 - An edge is added between each pair of peers which represent adjacent areas
 - Neighborhood information is stored by a peer in a **routing table**



... Content Addressable Network (CAN) ...

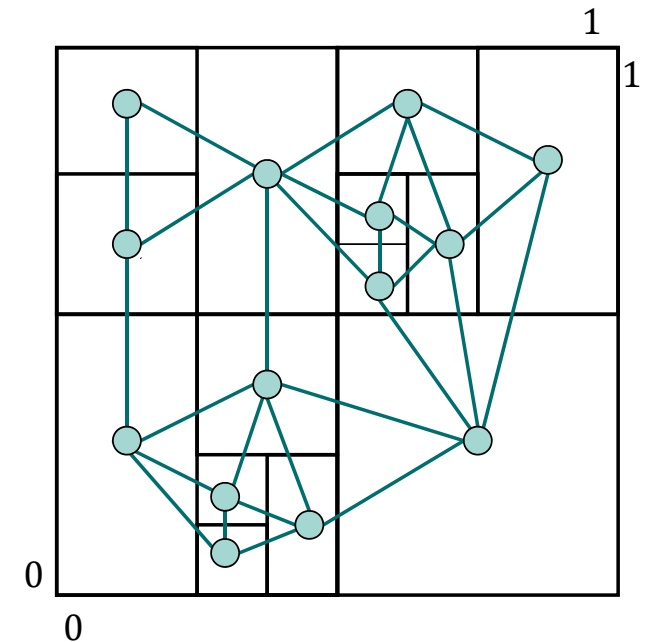
- What about the P2P Network?
- Each peer is responsible for an area in the d-dimensional space
- The overlay network is established as follows
 - Each peer is represented by a node
 - An edge is added between each pair of peers which represent adjacent areas
 - Neighborhood information is stored by a peer in a **routing table**



... Content Addressable Network (CAN) ...

Query processing in CANs

- For a given value, the hash function is applied and its coordinates in the d-dimensional space is calculated
- The query starts at the requesting peer. From there, it is routed from peer to peer towards the coordinates of the target value (by approximating the straight connection) – this is called **content-based routing**
- **Complexity of search:** $O(d \cdot n^{1/d})$
 - # of Hops in d dimensions: $O(n^{1/d})$
 - Routing information per peer: $O(d)$

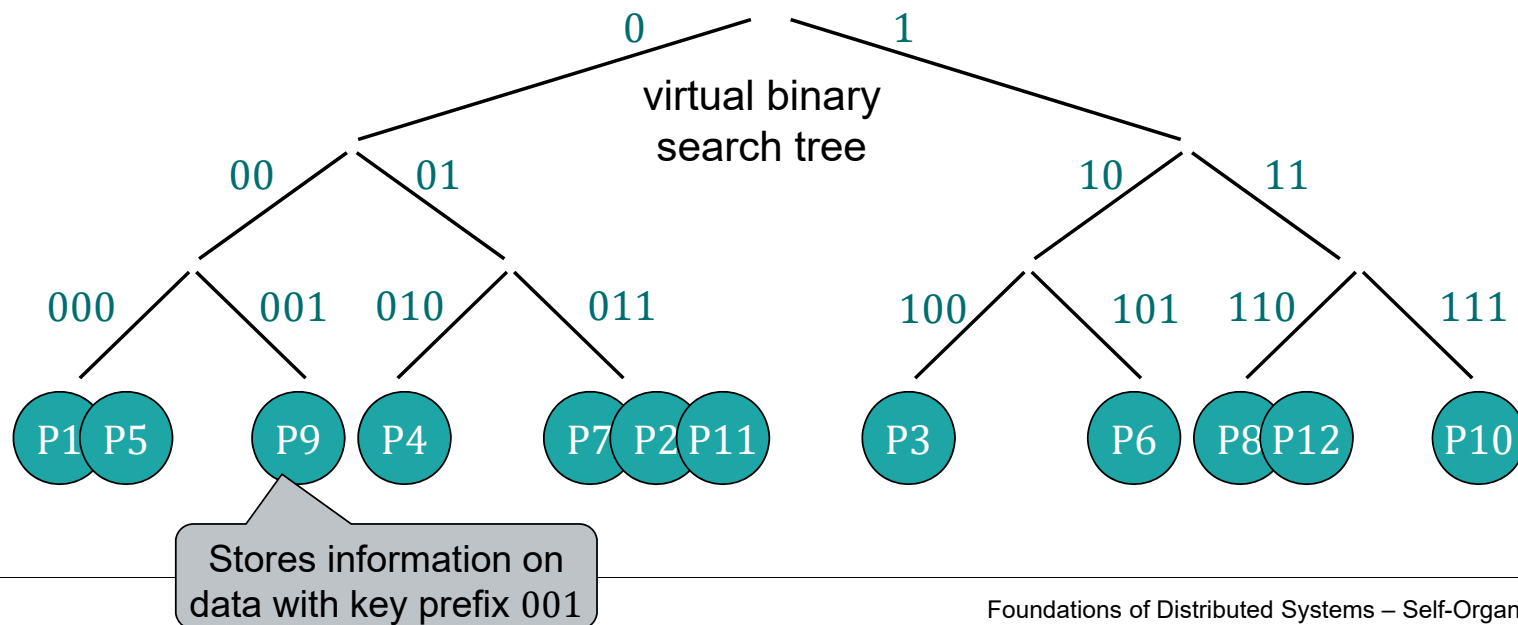


... Content Addressable Network (CAN)

- Properties of CAN
 - + CAN is **simple and robust mechanism** that balances the data to be stored
 - + The associated graph has a **low degree**, although the network is **strongly connected** and thus robust
 - + There are several ways to the target; thus the route can be optimized

P-Grid ...

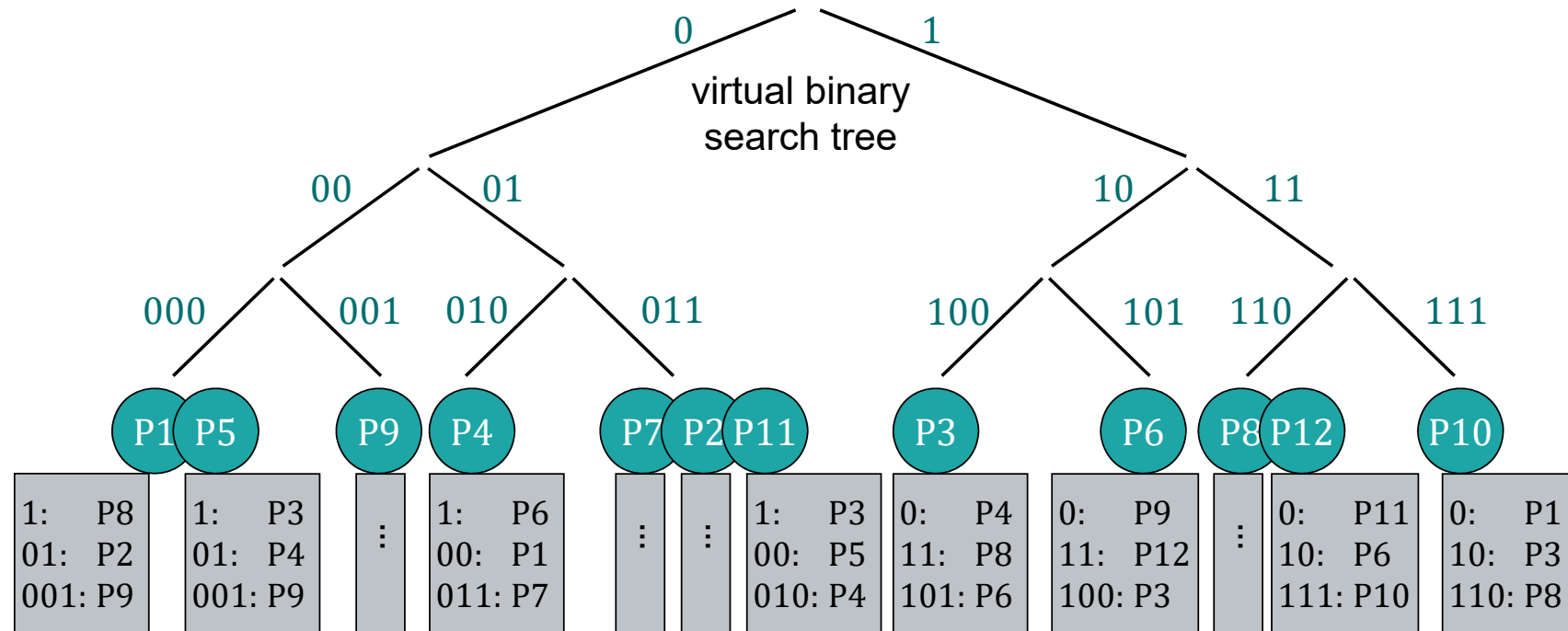
- Data are managed in a (virtual, distributed) search tree. The keys are binary values (i.e., the hash function maps a value to a binary key)
- Each peer p represents a node n in a tree. This means that p is responsible for all data in the sub-tree which is rooted at n
- There might be several peers responsible for the same node in the tree (redundancy)



... P-Grid ...

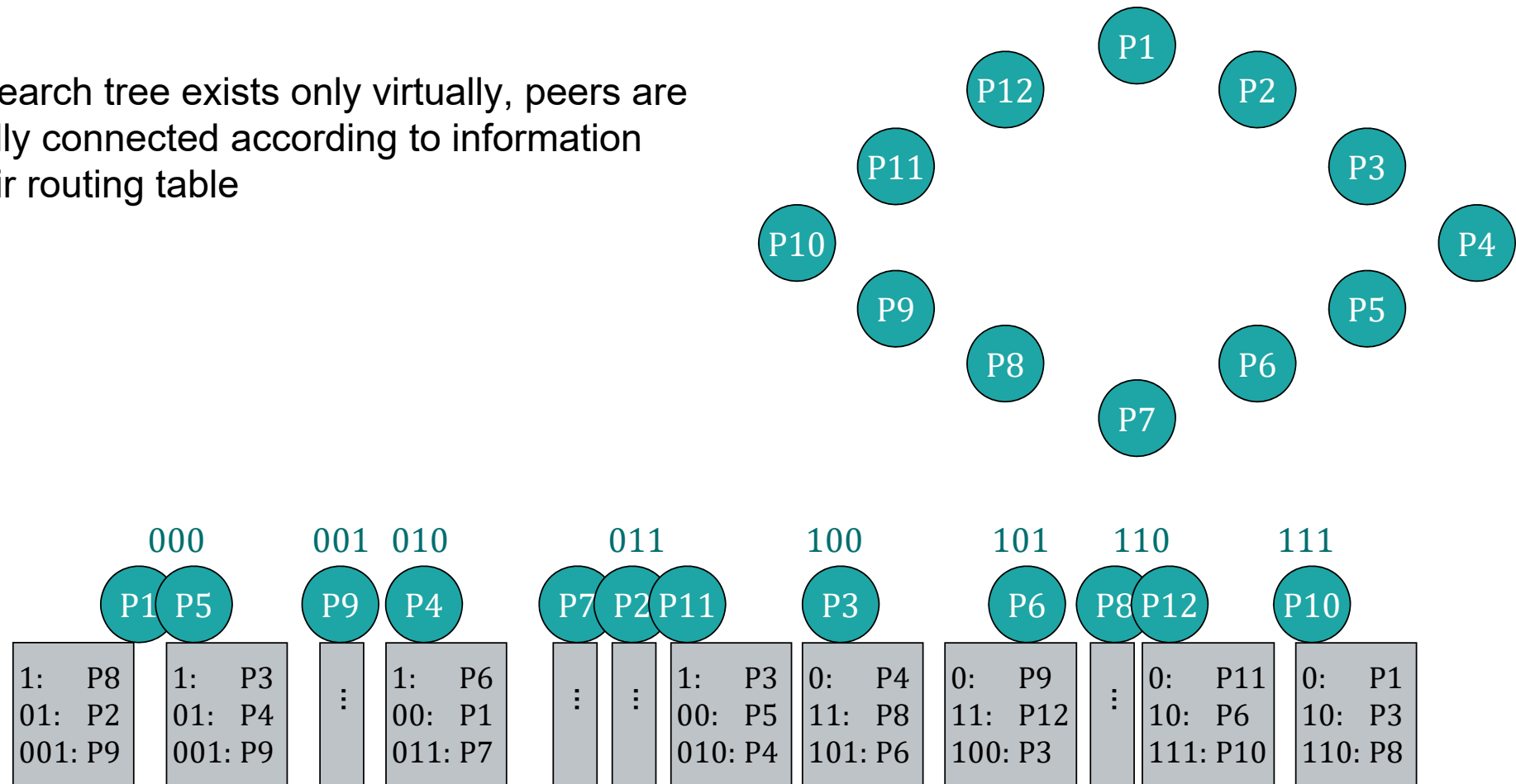
- A peer stores information according to its position in the search tree (the node corresponds to the prefix of all keys managed at the node)
- Thus, a peer can directly answer all queries where its node is a prefix of the key
- In order to support a query that cannot be supported by that peer (the data is not in its sub-tree), each peer has a routing table with information on at least one peer that is responsible for the 'other side of the tree' (one for each prefix)

... P-Grid ...



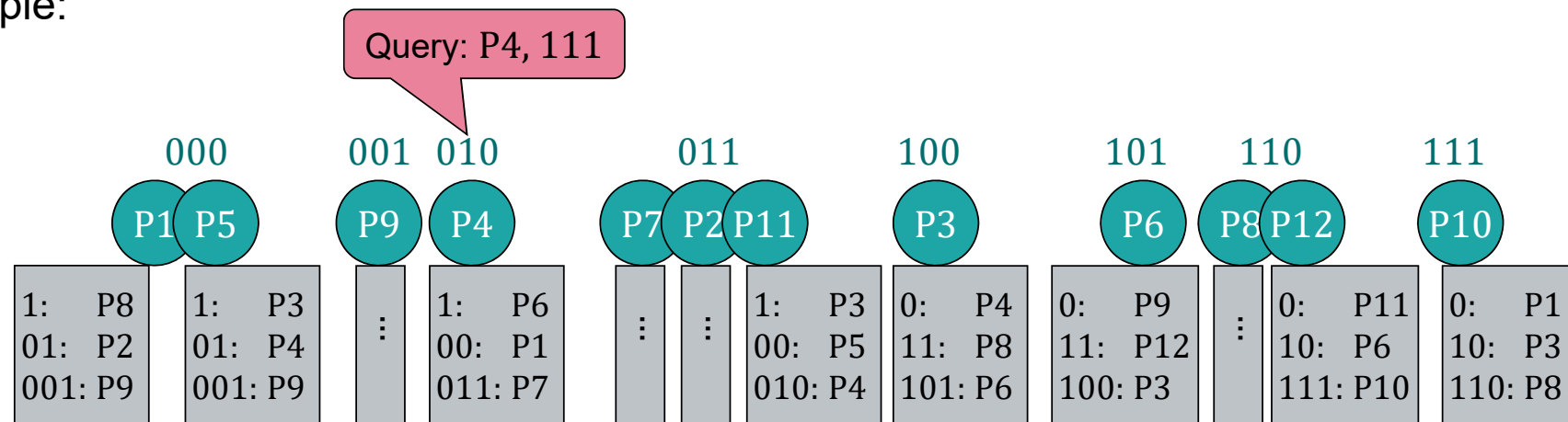
... P-Grid ...

- The search tree exists only virtually, peers are actually connected according to information in their routing table



... P-Grid

- Query processing: if a query comes to a peer, it either
 - Returns the data (in case it is responsible for)
 - Or it forwards the query to a peer which is closer to the data (on the 'other side' of the tree). This peer can either answer the query or it finds a peer which is even closer
- A query can be answered in m hops (with m being the height of the tree) complexity: $O(\log n)$, n : number of leaf nodes in the virtual binary tree
- Example:

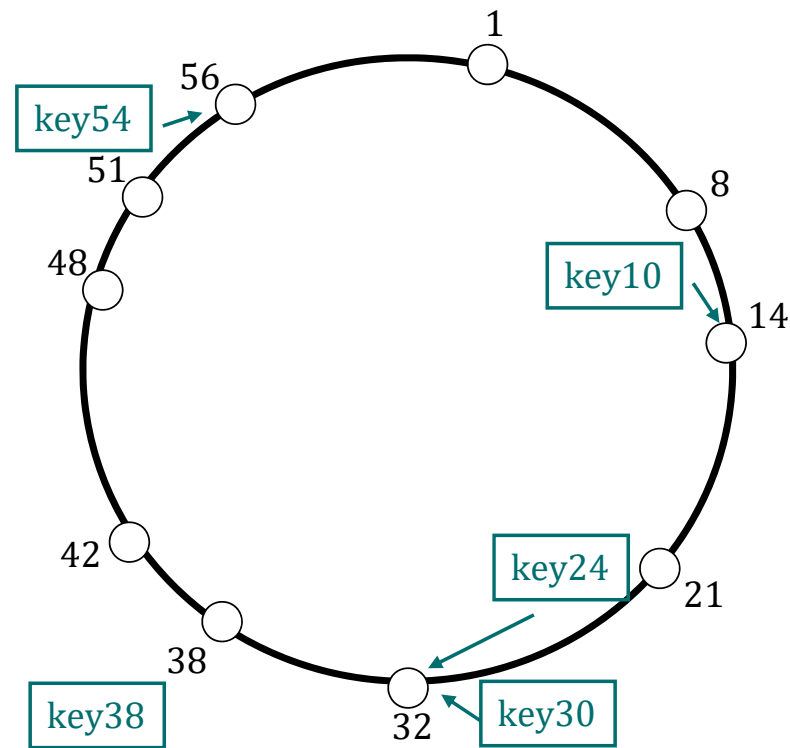


CHORD ...

- In CHORD, identifiers are organized in an **identifier circle, modulo 2^m** (so-called **CHORD ring**)
- Peers and data are mapped by a hash function to an identifier (key)
 - For peers, usually their IP address is chosen
 - For data, this is done by using their value
- The CHORD ring is able to manage at most 2^m peers, although much less is common (can actually exist) in the system
- A **value with key k is assigned the first peer with identifier $\geq k$** (called **successor node of key k**)

... CHORD ...

- Example: $m=6$, i.e., the identifier space is $[0, 63]$.
 - The CHORD ring encompasses 10 peers (circles) and stores 5 values

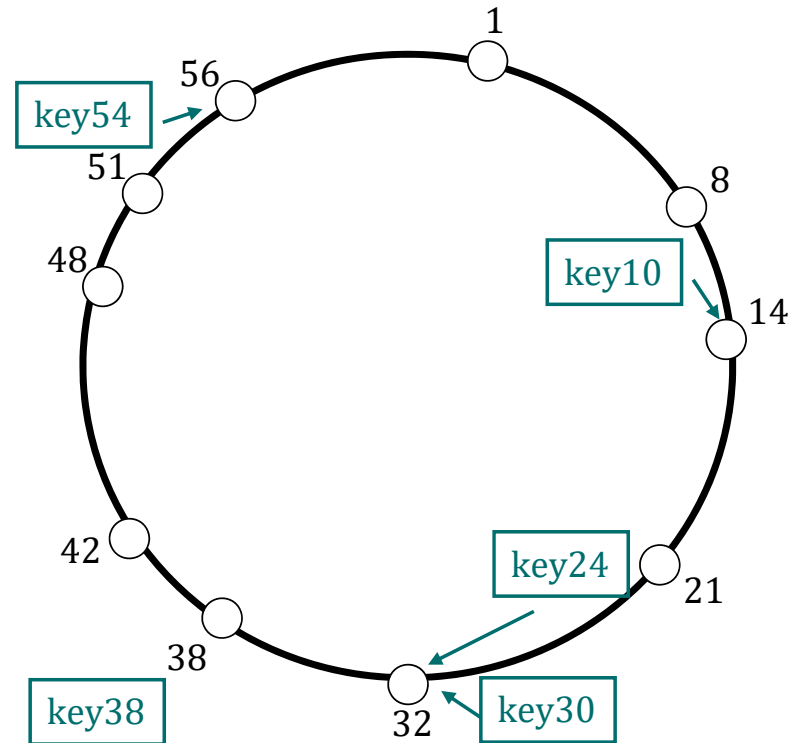


... CHORD ...

- Each peer knows at least its successor in the CHORD ring
- Queries for a key k_q have to be passed from one peer to its successor along the ring structure until the first peer with $ID \geq k_q$ is reached
- Then, either the value can be found with this peer or, if the value cannot be found there, it is not present
- Complexity is linear: $O(n)$

... CHORD ...

- Example: query with $k_q = 54$

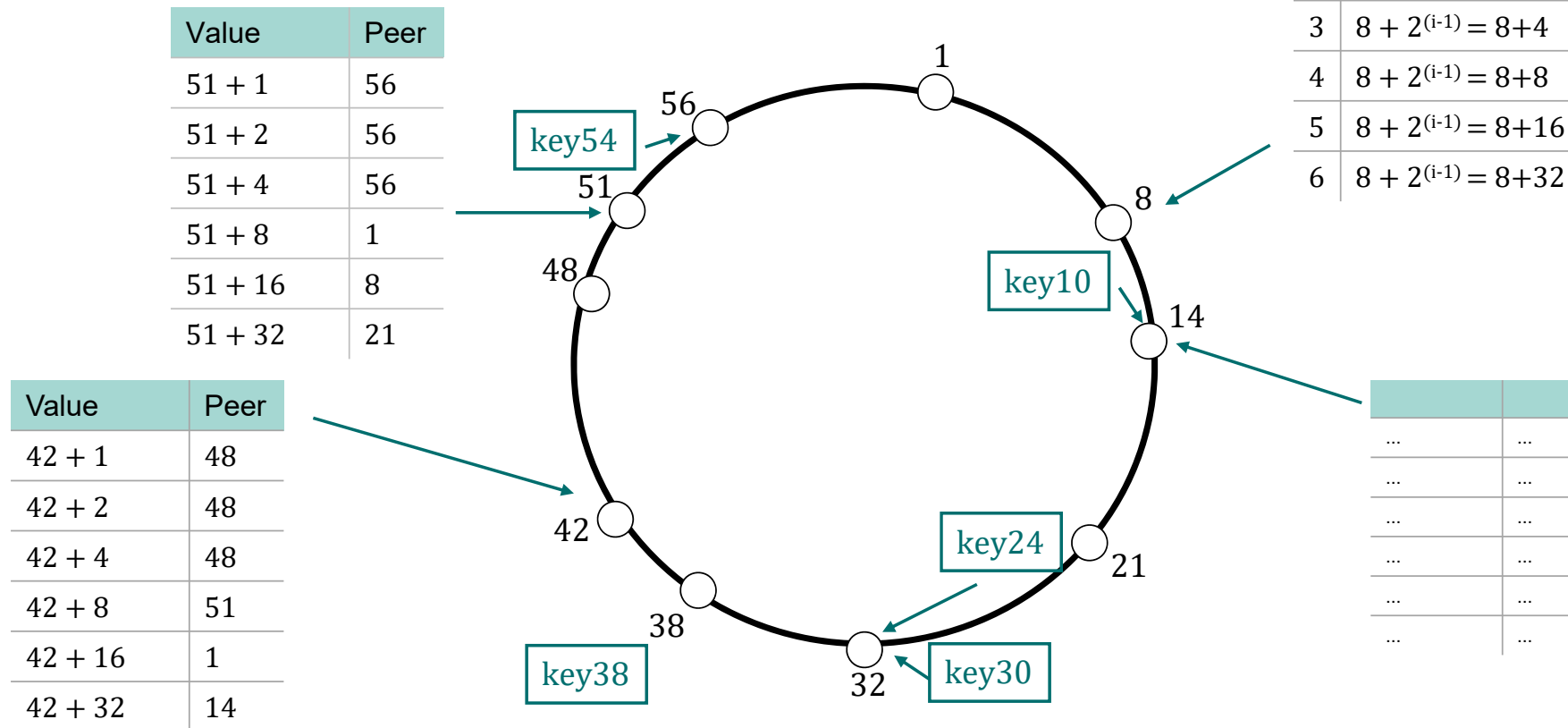


... CHORD ...

- The linear complexity can be significantly improved if each peer maintains a local routing table (so-called **finger table**)
 - The finger table contains (at most) m entries for an identifier space of 2^m
 - For a peer with identifier n , the finger table contains the addresses of the peers responsible for the following keys $(n+2^0) \bmod 2^m$ (= successor), $(n+2^1) \bmod 2^m$, $(n+2^2) \bmod 2^m$, ..., $(n+2^{m-1}) \bmod 2^m$
 - The finger table now allows to take 'shortcuts' through the ring to speed up queries

... CHORD ...

- Sample CHORD ring with finger tables



... CHORD ...

- The finger table not only contains information on the direct successor $n+2^0$ but also on other peers (more information on local keys, less information –and also less precise information– on more remote keys)
- When a query for key k_q arrives at a peer p_i , then p_i first checks if it can locally answer this query
 - If not, it checks in its local finger table the largest value k^* that precedes k_q
 - The query is directly forwarded to the peer p_j responsible for k^* .
This peer p_j is closer to k_q than p_i and thus contains more detailed information on how to find k_q
 - This is repeated until the key k_q is found (or until it is clear that key k_q does not exist)
 - Complexity of search: $O(\log n)$

... CHORD ...

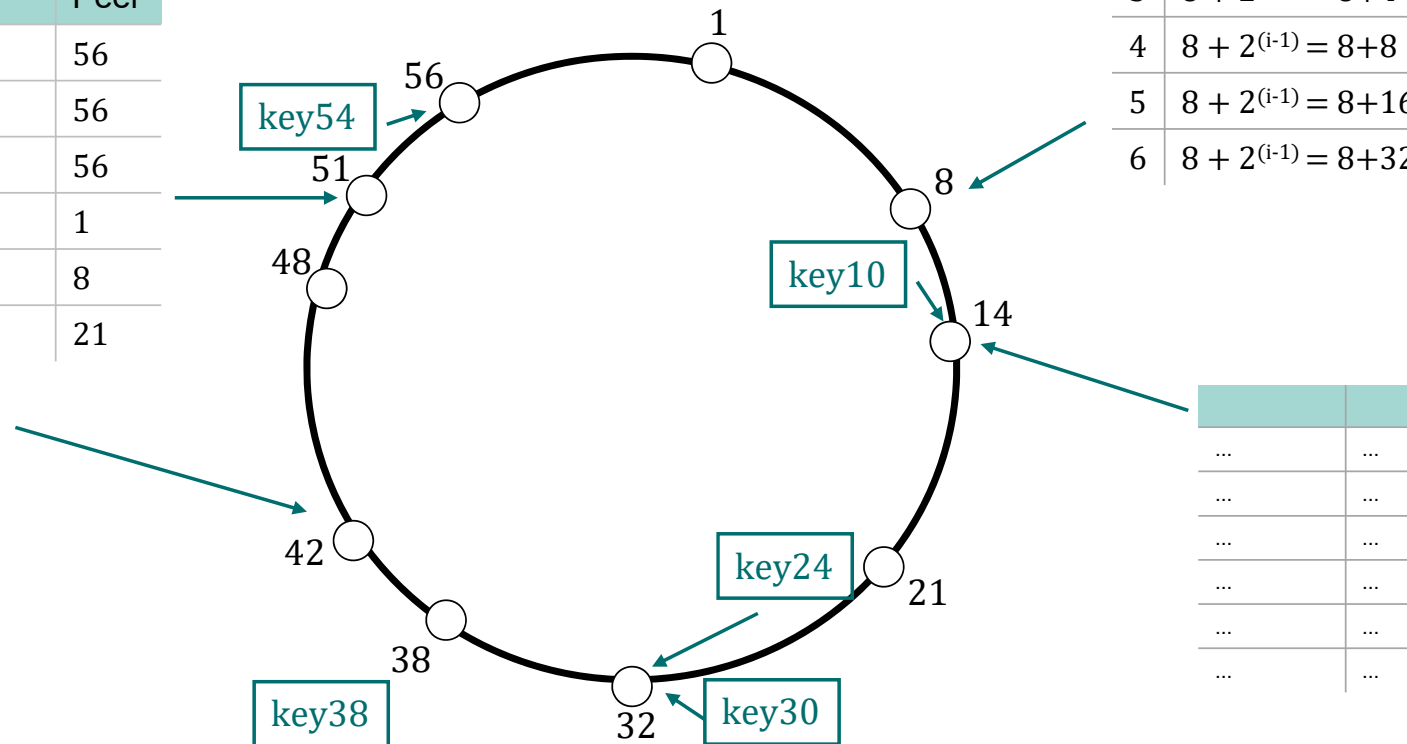
- Example: query $k_q = 0$ (sent to peer 8)

Finger table for peer 8

i	Value	Peer
1	$8 + 2^{(i-1)} = 8+1$	14
2	$8 + 2^{(i-1)} = 8+2$	14
3	$8 + 2^{(i-1)} = 8+4$	14
4	$8 + 2^{(i-1)} = 8+8$	21
5	$8 + 2^{(i-1)} = 8+16$	32
6	$8 + 2^{(i-1)} = 8+32$	42

Value	Peer
$51 + 1$	56
$51 + 2$	56
$51 + 4$	56
$51 + 8$	1
$51 + 16$	8
$51 + 32$	21

Value	Peer
$42 + 1$	48
$42 + 2$	48
$42 + 4$	48
$42 + 8$	51
$42 + 16$	1
$42 + 32$	14



Value	Peer
...	...
...	...
...	...
...	...
...	...
...	...

... CHORD ...

- In addition to the immediate successor (peer responsible for $\text{key} + 2^0$) and the entries in the finger table, each peer also maintains information on its predecessor p_p (needed especially for network dynamics, i.e., when peers join or leave, respectively)
- To make sure that the information in the finger table is correct, a peer p_i periodically sends **stabilize()** messages to its successor p_s
 - A peer q which receives a **stabilize()** message answers with its predecessor (q_p). Peer p_i then locally checks whether the finger table has to be updated (i.e., whether entry p_s [assumed successor] has to be replaced by q_p [actual successor])
 - Similarly, q checks whether p_i will become new predecessor
 - This is necessary if peer p_i has recently joined the ring and informs q about its presence
- Similarly, **fix_finger()** is used periodically to update the information in the finger tables

... CHORD ...

- When a new peer p_i joins, it has to contact an arbitrarily chosen peer p^* from the ring in order to get its successor p_s (similar to finding the peer responsible for the key that corresponds to the identifier of the new peer p_i)
 - Assume that p_i will be placed between peers p_p and p_s
- Peer p_i sends a **stabilize()** message to its new successor p_s
- Peer p_s updates its predecessor (p_i)
- Peers p_i and p_s re-distribute their data
 - all data with keys in the interval $]p_p, p_i]$ go to peer p_i , all keys in the interval $]p_i, p_s]$ stay with p_s
- When p_p uses **stabilize()** which will be sent to p_s (from which p_p assumes that it is its direct successor), it will answer with p_i
 - Peer p_p can update its successor (p_i instead of p_s)

... CHORD

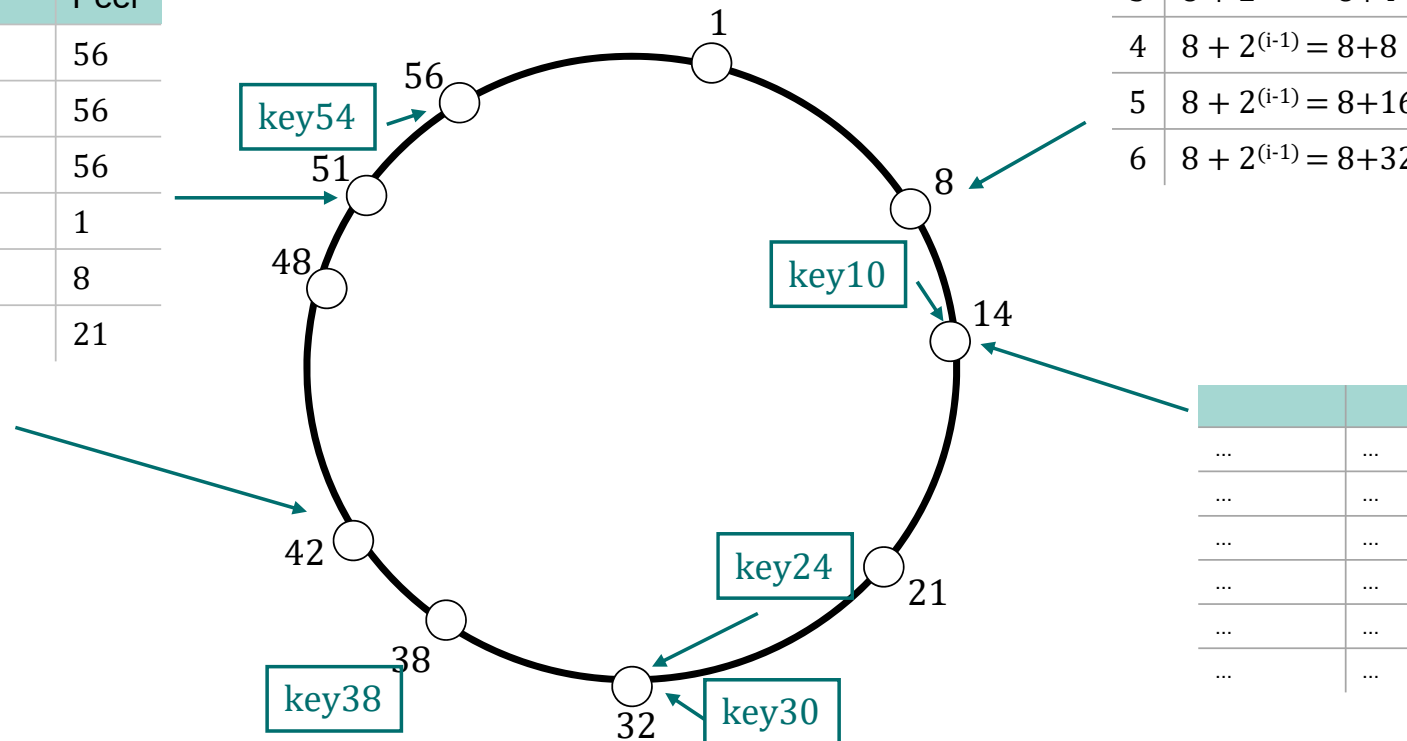
- Example: add peer with identifier 26

Finger table for peer 8

i	Value	Peer
1	$8 + 2^{(i-1)} = 8+1$	14
2	$8 + 2^{(i-1)} = 8+2$	14
3	$8 + 2^{(i-1)} = 8+4$	14
4	$8 + 2^{(i-1)} = 8+8$	21
5	$8 + 2^{(i-1)} = 8+16$	32
6	$8 + 2^{(i-1)} = 8+32$	42

Value	Peer
$51 + 1$	56
$51 + 2$	56
$51 + 4$	56
$51 + 8$	1
$51 + 16$	8
$51 + 32$	21

Value	Peer
$42 + 1$	48
$42 + 2$	48
$42 + 4$	48
$42 + 8$	51
$42 + 16$	1
$42 + 32$	14



Summary DHTs

	Search Paradigm	Search Type	Search cost (# of messages)
gnutella	Breadth-first search on graph	String comparison	$2 \cdot \sum_{i=0}^{\text{TTL}} C \cdot (C-1)^i$ (C: connections per peer)
CAN	Find region in d-dimensional space	Equality of key	$O(d \cdot n^{1/d})$
P-Grid	Binary prefix tree	Prefix of key (range)	$O(\log n)$
CHORD	Implicit binary search trees	Equality	$O(\log n)$

Recap: Questions

- What makes Napster a P2P system?
- What are the differences and commonalities between Napster and gnutella?
- Why are hash tables not well suited for dynamic distributed data management?
- What are the main concepts of *distributed* hash tables?
- What are the commonalities between CAN, P-Grid, and CHORD, what are the major differences between the three systems?
- How are neighborhoods defined in CAN, P-Grid, and CHORD?

Further Reading

- [Abe 01] K. Aberer: *P-Grid: A Self-Organizing Access Structure for P2P Information Systems*. In: Proceedings of the 6th International Conference on Cooperative Information Systems (CoopIS 2001), 2001.
- [RD 10] R. Rodrigues, P. Druschel: *Peer-to-Peer Systems*. Communications of the ACM, Vol. 53, No. 10, October 2010.
- [RFM⁺01] S. Ratnasamy, P. Francis, M. Handley, R. Karp, S. Shenker: *A Scalable Content-Addressable Network*. Proceedings of the ACM SIGCOMM, 2001.
- [SMK⁺01] I. Stoica, R. Morris, D. Karger, F. Kaashoek, H. Balakrishnan: *Chord: A Scalable Peer-To-Peer Lookup Service for Internet Applications*. Proceedings of the ACM SIGCOMM, 2001.